

Módulo 6 – Capítulo 09 – Sección 01

Diagnóstico de problemas: faithfulness baja, alucinaciones y respuestas incompletas

Un sistema RAG en producción que empieza a mostrar problemas de calidad puede fallar por al menos dos razones estructuralmente distintas: o el retriever no recupera los chunks que contienen la información necesaria, o el generador no usa correctamente los chunks que sí se recuperaron. La distinción parece obvia enunciada así, pero en la práctica la mayoría de los equipos aplica correcciones al primer componente que se les ocurre sin haber diagnosticado en cuál de los dos reside el problema. El resultado es una optimización inefectiva: mejorar el reranking no soluciona nada si el fallo está en el system prompt del generador, y reescribir las instrucciones del LLM no ayuda si el retriever nunca recupera el chunk que contiene la respuesta. Antes de tocar cualquier parámetro del sistema, el diagnóstico sistemático de la causa raíz es el paso más valioso e irremplazable.

La herramienta de diagnóstico fundamental es el análisis manual de 20 a 30 casos de respuestas de baja calidad, seleccionados del dataset de evaluación o de los registros de producción. Para cada caso, el analista examina simultáneamente la query original, los chunks recuperados y su orden, y la respuesta generada, respondiendo una pregunta central: "¿estaba la información necesaria para responder correctamente en los chunks recuperados?". Si la respuesta es sí, el fallo es del generador; si la respuesta es no, el fallo es del retriever. Esta categorización simple produce una distribución de tipos de error que revela dónde está concentrada la mayoría de los fallos en ese sistema específico, y esa distribución determina el roadmap de optimización.

Los cuatro tipos de error cubren la mayoría de los fallos observados en sistemas RAG reales. El **Tipo 1** (contexto correcto, respuesta incorrecta) ocurre cuando el chunk con la información está presente en el contexto pero el LLM no lo usa correctamente: el chunk relevante quedó enterrado en una posición central del contexto largo (el "lost-in-the-middle effect" documentado por Liu et al. 2023), el system prompt no instruye suficientemente al LLM para basar sus respuestas estrictamente en el contexto, o el modelo generador no tiene la capacidad de razonamiento necesaria para extraer la respuesta del chunk en el formato

requerido. El **Tipo 2** (contexto incompleto o incorrecto) ocurre cuando el retriever no devuelve los chunks que contienen la respuesta: la estrategia de chunking cortó la respuesta entre dos chunks sin solapamiento, el modelo de embedding no captura la similitud semántica entre la query y el documento relevante porque la query y el documento usan vocabulario distinto para referirse al mismo concepto, o el corpus simplemente no tiene un documento que cubra el tema preguntado. El **Tipo 3** (alucinación por ausencia de cobertura) es el más grave en términos de impacto en la confianza del usuario: el corpus no contiene información sobre lo que se pregunta, pero el LLM genera una respuesta de todos modos usando su conocimiento paramétrico en lugar de reconocer la ausencia de información. El **Tipo 4** (respuesta incompleta) ocurre cuando la query requiere integrar información de múltiples documentos y el retriever solo recupera algunos de los chunks necesarios, produciendo una respuesta parcialmente correcta.

Categorías de diagnóstico y acciones correctivas

- **Tipo 1 — fallo del generador:** presencia del chunk relevante verificada pero respuesta incorrecta; correcciones: reranking para mover el chunk más relevante a la primera posición del contexto, instrucciones explícitas en el system prompt ("Responde únicamente basándote en los documentos proporcionados a continuación"), cambio a un LLM de mayor capacidad; no cambiar nada del retriever.
- **Tipo 2 — fallo del retriever por chunking:** el chunk con la respuesta existe en el corpus pero fue cortado entre dos chunks sin solapamiento suficiente; correcciones: aumentar el overlap entre chunks, cambiar a chunking semántico o por estructura del documento, implementar parent-child indexing para recuperar el chunk padre con contexto completo.
- **Tipo 2 — fallo del retriever por embedding:** el chunk existe pero el modelo de embedding no lo recupera porque la query usa vocabulario distinto; correcciones: activar búsqueda híbrida (BM25 + vectorial) para capturar coincidencias léxicas, implementar HyDE o query expansion, evaluar un modelo de embedding diferente sobre el golden dataset del dominio.
- **Tipo 3 — alucinación por ausencia de cobertura:** correcciones: implementar un score threshold de similitud (similarity score del top chunk $< 0.6 \rightarrow$ respuesta de no-información en lugar de generación libre), instrucciones explícitas en el system prompt para responder "No tengo información suficiente en los documentos disponibles" cuando la información no está presente, guardrail de faithfulness post-generación.

- **Tipo 4 — respuesta incompleta por cobertura parcial:** correcciones: aumentar K (recuperar más chunks por query), implementar multi-query retrieval para generar múltiples formulaciones de la query, considerar GraphRAG si las queries de síntesis son frecuentes en el sistema.
- **Diagnóstico con RAGAS:** ejecutar el framework RAGAS sobre el dataset de errores; bajo Context Recall + baja Faithfulness indica problema del retriever (los chunks correctos no están llegando al LLM); alto Context Recall + baja Faithfulness indica problema del generador (los chunks correctos llegan pero el LLM no los usa bien); bajo Answer Relevancy con alta Faithfulness indica problema de system prompt o capacidad del modelo generador.

Nota del Arquitecto: El error más frecuente en la optimización de sistemas RAG es saltar directamente a ajustar parámetros sin el análisis manual previo. El análisis de 20–30 casos de error toma 2–4 horas, pero puede evitar semanas de trabajo de optimización aplicado al componente equivocado. En sistemas donde el 70% de los fallos son de Tipo 2 (retriever), mejorar el system prompt del generador no produce ninguna mejora medible en las métricas de evaluación. La inversión en diagnóstico tiene el ROI más alto de cualquier actividad de optimización en RAG.

El diagnóstico preciso de la distribución de tipos de error establece el programa de optimización: las secciones siguientes presentan las técnicas específicas para optimizar el retriever, comprimir el contexto, reducir el costo operativo y medir el impacto de los cambios con rigor estadístico.

Módulo 6 – Capítulo 09 – Sección 02

Optimización del retriever: ajuste de K, reranking y filtros de relevancia

Una vez que el diagnóstico confirma que los fallos principales son del retriever —los chunks con la información no están entre los recuperados— la optimización debe ser sistemática y empírica: cambiar un parámetro a la vez, medir el impacto sobre el golden dataset completo, y acumular las mejoras que superan el umbral de significancia estadística. Optimizar el retriever "a intuición" —subiendo K porque parece razonable, activando el reranker porque reduce la sensación de que los chunks no son relevantes— es una práctica que puede parecer que mejora la calidad sin que ninguna métrica lo confirme. El golden dataset con sus métricas de Recall@K y Precision@K es el árbitro objetivo de si un cambio produce mejora real o no.

El parámetro **K** es el punto de partida de la optimización del retriever por su impacto directo en el Recall@K. Aumentar K de 3 a 10 casi siempre mejora el Recall: hay más oportunidades de que el chunk relevante esté entre los recuperados. Pero el costo de ese aumento es doble: el contexto que el LLM debe procesar crece proporcionalmente (con su impacto en latencia y costo de inferencia), y el "lost-in-the-middle" effect se intensifica cuando hay 10 chunks en el contexto en lugar de 3. La estrategia correcta es trazar la curva de Recall@K midiendo el Recall para K = 1, 3, 5, 10, 15 y 20 sobre el golden dataset, e identificar el "knee" de la curva: el punto donde el incremento marginal de Recall por unidad adicional de K se vuelve pequeño. Ese punto es el K óptimo para el sistema específico. En sistemas de QA factual donde la respuesta está en un único chunk bien delimitado, el knee suele estar en K = 3–5. En sistemas de síntesis que requieren integrar información de múltiples documentos, el knee puede estar en K = 15–20.

El **reranking** es consistentemente la optimización de mayor retorno sobre la inversión en calidad del retriever. En lugar de mejorar el sistema de recuperación inicial, el reranking corrige su ordenamiento con un modelo más preciso pero más costoso. El pipeline two-stage —recuperar los top-100 chunks con búsqueda vectorial rápida y luego reordenarlos con un cross-encoder que evalúa cada par (query, chunk) de forma independiente— mejora la Precision@5 en 15–25 puntos porcentuales respecto al pipeline sin reranker. La razón técnica

es que el cross-encoder puede evaluar la relevancia de un chunk específico para una query específica con una profundidad de comprensión que un bi-encoder no alcanza, porque el bi-encoder produce embeddings independientes de la query y el chunk y mide su similitud por producto escalar, mientras el cross-encoder tiene acceso simultáneo a ambos textos y puede evaluar su interacción semántica completa. El overhead de latencia del reranker (200–500ms para procesar 100 chunks) se amortiza sobre el beneficio de Precision@5 para la mayoría de los sistemas de producción.

El **umbral de relevancia** es una técnica simple que mejora la Faithfulness del sistema al reducir el ruido en el contexto. Cuando el top chunk recuperado tiene un score de similitud coseno muy bajo (por debajo de 0.60–0.75 según el modelo de embedding), es probable que el corpus no contenga información relevante para la query y que todos los chunks recuperados sean los "menos malos" de un corpus que no cubre el tema. En ese caso, incluir esos chunks en el contexto del LLM frecuentemente produce alucinaciones: el LLM recibe chunks que se aproximan al tema pero no lo responden, y tiende a extrapolar o inventar la información faltante. Rechazar chunks por debajo del umbral de relevancia y responder "No tengo información suficiente en el corpus" cuando todos los chunks caen por debajo del umbral reduce la tasa de alucinaciones de forma significativa.

Técnicas de optimización del retriever

- **Curva Recall@K y punto óptimo de K:** medir Recall@1, @3, @5, @10, @15, @20 sobre el golden dataset; identificar el knee de la curva (máximo Recall con mínimo costo de contexto); no asumir K=5 por defecto — el K óptimo es específico del tipo de queries del sistema.
- **Reranking two-stage:** first-stage con búsqueda vectorial/híbrida recuperando top-50 o top-100 chunks; second-stage con cross-encoder (Cohere Rerank v3 a \$2/M, BGE Reranker v2-m3, FlashRank); medir Precision@5 antes y después; mejoras típicas de 15–25 puntos porcentuales; la inversión en latencia de 200–500ms suele justificarse.
- **Score threshold para no-respuesta:** configurar un umbral de score mínimo del top chunk (0.60–0.75 según el modelo de embedding); si el top chunk está por debajo del umbral, responder con un mensaje de ausencia de información en lugar de pasar chunks de baja relevancia al generador; calibrar el umbral sobre el dataset de evaluación con queries sin respuesta en el corpus.
- **Maximal Marginal Relevance (MMR):** seleccionar los K chunks que maximizan la relevancia con la query y minimizan la redundancia entre chunks seleccionados; útil cuando múltiples chunks del corpus son casi idénticos y el retriever tiende a recuperar

versiones redundantes del mismo contenido; implementado en LangChain con `MMRRetriever`; el parámetro `lambda` controla el balance entre relevancia y diversidad.

- **Alpha dinámico para búsqueda híbrida:** ajustar el peso del componente `dense` vs. `sparse` según el tipo de query detectado; queries con entidades nombradas específicas, códigos de producto o términos técnicos exactos se benefician de más peso a BM25 (alpha bajo); queries conceptuales y de similitud semántica se benefician de más peso al vectorial (alpha alto); un clasificador simple de tipo de query puede ajustar alpha dinámicamente.
- **Fine-tuning del modelo de embedding:** cuando el análisis de errores muestra que el retriever falla sistemáticamente para un tipo de query o dominio del corpus, generar triplas contrastivas (`query`, `chunk_positivo`, `chunk_negativo_difícil`) del corpus de dominio y hacer fine-tuning del modelo base con `sentence-transformers`; mejoras de 10–20 puntos de `Recall@5` en dominios muy especializados donde los modelos preentrenados no capturan bien la terminología técnica.

Nota del Arquitecto: La combinación de *K* óptimo + reranking + umbral de relevancia, cada uno calibrado sobre el golden dataset de forma independiente, puede acumular una mejora de 20–35 puntos de `Precision@5` respecto al pipeline con parámetros por defecto. Es la combinación de optimizaciones de retriever con mejor ROI disponible hoy. Medir el impacto de cada componente por separado antes de combinarlos permite atribuir la mejora a la técnica correcta y saber qué ocurre si en el futuro se necesita simplificar el pipeline.

Con el retriever optimizado para maximizar la calidad de los chunks entregados al generador, el siguiente desafío es gestionar cuántos de esos chunks —y cuánto de cada uno— incluir en el contexto de la generación sin exceder los límites de costo y latencia.

Módulo 6 – Capítulo 09 – Sección 03

Compresión de contexto: reducir tokens sin perder información relevante

Hay un principio en los sistemas de recuperación de información que no desaparece con el RAG: más información en el contexto no siempre produce respuestas de mayor calidad. En el caso específico de los LLMs, la relación entre longitud del contexto y calidad de la respuesta no es monotonía: hasta cierto punto, más contexto aporta más información y mejora la respuesta; más allá de ese punto, el contexto adicional introduce ruido que distrae al modelo, aumenta la probabilidad del lost-in-the-middle effect (donde el LLM pierde capacidad de atención sobre chunks ubicados en posiciones centrales de un contexto muy largo), y escala el costo de inferencia de forma lineal con el número de tokens. La compresión de contexto es el conjunto de técnicas que maximizan la información relevante dentro de un presupuesto de tokens, permitiendo un K mayor sin el costo de un contexto proporcional.

El problema central que la compresión aborda es la **heterogeneidad del contenido** dentro de cada chunk recuperado. Un chunk de 500 tokens sobre la política de devoluciones de una empresa contiene, probablemente, 50 tokens con la información directamente relevante para la query del usuario ("¿cuántos días tengo para devolver un producto defectuoso?"), 200 tokens de contexto útil que enriquece la respuesta, y 250 tokens de información administrativa, fechas de versión, referencias cruzadas y cláusulas generales que son irrelevantes para esa query específica. El LLM debe procesar los 500 tokens de forma completa, pero su respuesta podría haberse generado con solo los 250 tokens relevantes. La compresión identifica y extrae esa fracción relevante de cada chunk antes de enviarlo al LLM generador.

LLMChainExtractor de LangChain es la implementación más directa de este principio: usa un LLM pequeño y económico —GPT-4o-mini a \$0.15 por millón de tokens de entrada, o Claude 3 Haiku a \$0.25 por millón de tokens— para extraer de cada chunk solo las oraciones o párrafos que son directamente relevantes para la query del usuario. El proceso es: enviar la query + el texto completo del chunk al LLM de compresión con la instrucción "extrae solo las frases directamente relevantes para responder la siguiente pregunta", obtener el texto

comprimido, y enviarlo al LLM generador. La reducción de tokens suele ser del 40–70%. El costo de la compresión (una llamada adicional al LLM por chunk) se amortiza cuando el sistema usa un LLM costoso como generador: comprimir 5 chunks de 500 tokens a 150 tokens promedio reduce el costo de input tokens del generador en un 70%, más que compensando el costo de las 5 llamadas de compresión con GPT-4o-mini. El overhead de latencia (200–400ms por chunk) es el principal inconveniente para sistemas con SLOs de latencia estrictos.

EmbeddingsFilter es la alternativa basada en similitud vectorial sin latencia de LLM adicional: en lugar de usar un LLM para extraer las oraciones relevantes, calcula el embedding de cada oración individual del chunk y compara su similitud coseno con el embedding de la query. Solo las oraciones que superan un umbral de similitud (típicamente 0.6–0.75) se incluyen en el contexto comprimido. El proceso es más rápido que LLMChainExtractor (no requiere una llamada a un LLM adicional, solo cómputo de embeddings que en muchos sistemas ya está disponible), pero con menor precisión: la similitud coseno no siempre captura correctamente qué oraciones son necesarias para responder una pregunta cuando la relevancia es inferencial en lugar de directamente semántica.

LongContextReorder es una estrategia que no comprime el número de tokens pero mejora la calidad de la atención del LLM sobre el contexto: reordena los chunks para colocar los más relevantes en la primera y en la última posición del contexto (donde la atención del modelo es más alta según Liu et al. 2023), y los menos relevantes en las posiciones centrales. Esta técnica no reduce el costo pero puede mejorar Faithfulness y Answer Relevancy en 5–10 puntos sin ningún overhead computacional adicional, y es compatible con cualquier estrategia de compresión.

Técnicas de compresión de contexto

- **LLMChainExtractor**: LLM pequeño (GPT-4o-mini, Claude 3 Haiku) extrae solo las oraciones relevantes de cada chunk; reducción de tokens del 40–70%; overhead de latencia de 200–400ms por chunk; costo neto favorable cuando el LLM generador es costoso (GPT-4o, Claude 3.5 Sonnet); implementado en LangChain como `ContextualCompressionRetriever` con `LLMChainExtractor`.
- **EmbeddingsFilter**: similitud coseno entre embedding de la query y embedding de cada oración del chunk; retener oraciones con similitud > umbral (0.60–0.75); más rápido que LLMChainExtractor, menor precisión en relevancia inferencial; sin overhead de llamada LLM adicional.

- **LongContextReorder:** reordenar chunks con el más relevante en posición 1 y posición K, el menos relevante en el centro; no reduce tokens pero mejora la atención del LLM sobre los chunks más importantes; sin costo adicional; implementado en LangChain como `LongContextReorder`.
- **Token budget management:** definir presupuesto máximo de tokens de contexto (3000–4000 tokens es un rango práctico para modelos con ventana de 8K); después del reranking, seleccionar los chunks que caben dentro del presupuesto en orden de relevancia; si el chunk más importante no cabe completo, comprimir ese chunk con LLMChainExtractor hasta que quepa en el presupuesto restante.
- **Sentence-level retrieval con agregación:** indexar a nivel de oración (1–2 oraciones por chunk) para máxima granularidad de recuperación; agregar las oraciones recuperadas en grupos coherentes por documento de origen para generar el contexto final; combina la precisión de la recuperación granular con la coherencia necesaria para el LLM generador.
- **Sliding window para documentos largos:** para queries que requieren información que puede estar en cualquier parte de un documento muy largo (manual de 200 páginas), evaluar múltiples ventanas del documento con un modelo de relevancia ligero y seleccionar la ventana de mayor relevancia; alternativa al chunking arbitrario cuando el corte puede partir la respuesta entre dos chunks.

Nota del Arquitecto: La compresión de contexto tiene el mayor impacto cuando el sistema tiene restricciones de costo por inferencia (alto volumen de queries con LLM costoso) o restricciones de latencia incompatibles con K grande. Antes de implementarla, verificar dos condiciones: primero, que la compresión no está eliminando información necesaria para responder la query (medir Faithfulness y Context Recall con y sin compresión sobre el golden dataset); segundo, que el ahorro en tokens del generador supera el costo adicional de las llamadas al LLM de compresión. Si ambas condiciones se cumplen, la compresión es una optimización sólida para sistemas de producción con alto volumen.

La optimización del retriever y la gestión del contexto abordan la calidad de las respuestas. La siguiente sección aborda la dimensión económica del sistema: cómo reducir el costo operativo sin comprometer la calidad medida en el golden dataset.

Módulo 6 – Capítulo 09 – Sección 04

Optimización de costos: modelo de generación, batch inference, caché y RAG conversacional

El costo operativo de un sistema RAG en producción no se percibe durante el prototipo, donde el volumen de queries es bajo y los experimentos se realizan manualmente. Se percibe cuando el sistema alcanza cientos o miles de queries diarias y los costos de inferencia del LLM y del modelo de embedding aparecen en la factura mensual de la API. Un sistema RAG con arquitectura de referencia —top-K=5 con chunks de 512 tokens, modelo de embedding a \$0.06/M tokens, LLM generador a \$3/M tokens de input + \$15/M de output, reranker con Cohere a \$2/M— tiene un costo por query de aproximadamente \$0.004–0.008 para queries típicas. A 10.000 queries diarias, ese costo acumula entre \$40 y \$80 por día, o entre \$1.200 y \$2.400 por mes. El escenario es manejable para sistemas empresariales de un equipo, pero una plataforma de soporte de alto volumen puede multiplicar esas cifras por 10 o 100, convirtiendo el costo en el factor limitante de la viabilidad del sistema.

La **selección del LLM generador** es la decisión de mayor impacto en el costo, con diferencias de precio de uno a dos órdenes de magnitud entre modelos. GPT-4o cuesta \$5 por millón de tokens de entrada y \$15 por millón de salida; GPT-4o-mini cuesta \$0.15 por millón de entrada y \$0.60 por millón de salida —una diferencia de 33x en entrada y 25x en salida. Claude 3.5 Sonnet cuesta \$3/M de entrada y \$15/M de salida; Claude 3 Haiku cuesta \$0.25/M de entrada y \$1.25/M de salida —diferencia de 12x en entrada. La pregunta crítica es: ¿en qué casos de uso la diferencia de calidad entre el modelo caro y el modelo económico es perceptible y medida? En sistemas RAG donde el contexto recuperado es de alta calidad y la tarea del generador es relativamente mecánica (extraer y reformular información del contexto, no razonar sobre casos ambiguos), GPT-4o-mini o Claude 3 Haiku producen respuestas de calidad indistinguible de GPT-4o o Claude 3.5 Sonnet para el 70–80% de las queries. La estrategia de **routing de modelo por complejidad** explota esto: clasificar cada query como simple (respuesta directa de un único chunk, sin ambigüedad) o compleja (síntesis de múltiples documentos, razonamiento multi-step, query ambigua que requiere interpretación), y usar el modelo económico para las simples y el modelo premium para las

complejas. Un clasificador ligero (regla basada en longitud de query y presencia de ciertas palabras clave, o un modelo de clasificación de 70M parámetros) puede reducir el uso del modelo premium al 20–30% del tráfico.

El **prompt caching de Anthropic** es una optimización de costo específica para Claude que puede reducir el costo de los tokens de sistema en un 90%. Claude 3.5 Sonnet y Claude 3 Haiku soportan cache breakpoints: si el system prompt es estático (las instrucciones no cambian entre queries), marcarlo con un breakpoint de caché hace que solo la primera solicitud pague el precio de procesamiento completo (\$3/M tokens); las solicitudes subsiguientes que reutilizan el mismo system prompt pagan solo \$0.30/M tokens (el 10% del precio original) para los tokens cacheados. En un sistema RAG donde el system prompt tiene 500 tokens de instrucciones fijas más los chunks del contexto (que sí cambian entre queries), el cache breakpoint después de las instrucciones fijas ahorra el 90% del costo de esos 500 tokens en todas las queries excepto la primera. A 10.000 queries diarias con system prompt de 500 tokens, el ahorro es de aproximadamente \$13 por día solo de este componente.

El **batch inference** es relevante para casos de uso que no requieren respuesta en tiempo real: generación de resúmenes de documentos para el corpus, indexación de preguntas hipotéticas para HyDE, evaluación masiva de respuestas con LLM-as-judge, o pipelines de enriquecimiento de chunks. OpenAI Batch API ofrece el 50% de descuento sobre los precios estándar (GPT-4o-mini a \$0.075/M tokens) para jobs procesados en una ventana de 24 horas. Anthropic Message Batches ofrece el 50% de descuento sobre los precios de Claude (Claude 3.5 Sonnet a \$1.50/M tokens de entrada) con el mismo modelo de ventana asíncrona. Para un pipeline de ingesta que procesa 10.000 documentos nuevos por semana con generación de contexto (Contextual Retrieval), el batch inference puede reducir el costo de enriquecimiento a la mitad.

Un patrón de optimización que merece especial atención por su prevalencia en sistemas empresariales maduros es el **RAG conversacional multi-turno**. Cuando el sistema sirve sesiones de conversación en lugar de queries independientes, cada turno adicional de la conversación incrementa el contexto del historial que debe enviarse al LLM en la siguiente query. Una sesión de 10 turnos con responses promedio de 200 tokens produce un historial de 2.000 tokens que se acumula al contexto de los chunks recuperados en cada turno subsiguiente. El costo escala linealmente con la longitud de la sesión. La solución de compresión de historial conversacional utiliza un LLM pequeño para resumir periódicamente el historial de conversación: en lugar de enviar los 10 turnos completos, enviar un resumen de 300 tokens que capture los acuerdos, restricciones y contexto establecido en la conversación.

Este patrón mantiene la coherencia conversacional —el sistema recuerda lo que el usuario preguntó antes y los compromisos tomados— mientras controla el crecimiento del costo por sesión. La frecuencia de resumen depende del SLO de costo: resumir cada 5 turnos es un balance práctico para la mayoría de los casos.

Estrategias de optimización de costos

- **Routing de modelo por complejidad:** clasificar queries en simples/complejas con un clasificador ligero; usar modelo económico (GPT-4o-mini, Claude 3 Haiku) para el 70–80% del tráfico simple; modelo premium para queries complejas; medir $\text{Quality@complejidad}$ para verificar que las queries simples no degradan con el modelo económico.
- **Prompt caching de Anthropic:** usar cache breakpoints en Claude 3.5 Sonnet y Haiku para system prompts estáticos; los tokens cacheados cuestan el 10% del precio estándar; ahorro de ~90% en el costo de los tokens de instrucción fija para todas las queries excepto la primera del caché.
- **Batch inference:** OpenAI Batch API (50% descuento, ventana de 24h) y Anthropic Message Batches (50% descuento) para workflows asíncronos: generación de contexto en ingesta (Contextual Retrieval), evaluación masiva con LLM-judge, generación de preguntas hipotéticas para HyDE; no usar en el path síncrono de respuesta al usuario.
- **Caching semántico:** queries repetidas (FAQ, preguntas de soporte frecuentes) servidas desde caché eliminan el costo de embedding + búsqueda + reranking + generación; cache hit rate del 30–50% en sistemas de soporte reduce el costo total proporcionalmente (ver Capítulo 7, Sección 4 para la implementación completa).
- **Reducción de tokens de contexto:** el costo de input tokens escala directamente con $K \times \text{chunk_size}$; $K=3$ con chunks de 256 tokens = 768 tokens de contexto; $K=10$ con chunks de 512 tokens = 5.120 tokens de contexto (6.7x diferencia de costo); optimizar K al mínimo que mantiene el Recall@K objetivo.
- **Compresión de historial conversacional:** en RAG multi-turno, resumir el historial de conversación cada 5 turnos con un LLM pequeño; el resumen de 300 tokens reemplaza 2.000–3.000 tokens de historial completo; mantiene coherencia conversacional con un 70–85% de reducción en tokens de historial.
- **Selección de modelo de embedding por volumen:** text-embedding-3-small (\$0.02/M tokens) vs. voyage-3 (\$0.06/M tokens) — diferencia de 3x; para corpus de millones de documentos reindexados frecuentemente, la diferencia es de decenas o cientos de dólares; evaluar si text-embedding-3-small mantiene Recall@K aceptable en el dominio específico antes de optar por modelos más caros.

Nota del Arquitecto: El tracking de costos desglosado por componente es el prerequisite de cualquier optimización económica efectiva. Un dashboard que muestra el costo diario separado en: embedding de queries, búsqueda vectorial (si es managed), reranking, generación (input + output tokens), permite identificar el componente de mayor gasto. En sistemas de bajo volumen de ingesta y alto volumen de queries, el generador domina el costo; en sistemas de alto volumen de corpus que se reindexa frecuentemente, el embedding de documentos puede ser el componente más costoso. Las estrategias de optimización correctas son distintas en cada caso.

Con las estrategias de calidad (retriever optimizado, compresión de contexto) y de costo (modelo routing, caching, batch inference) en su lugar, la última pieza del sistema de optimización es el método científico para comparar configuraciones en producción: el A/B testing.

Módulo 6 – Capítulo 09 – Sección 05

A/B testing en sistemas RAG: comparación de configuraciones en producción

La optimización de un sistema RAG en producción enfrenta un problema epistemológico que no existe en el desarrollo inicial: cómo saber con certeza estadística si un cambio en la configuración del pipeline produce una mejora real en la experiencia de los usuarios reales, o si lo que se observa es variabilidad aleatoria del sistema. En desarrollo, el golden dataset provee un benchmark estable que permite comparar configuraciones. Pero el golden dataset tiene limitaciones: fue construido en un momento específico con una muestra de queries que puede no representar perfectamente la distribución real de uso del sistema, y no captura los cambios sutiles en la experiencia del usuario que solo emergen en producción (velocidad percibida, coherencia conversacional, satisfacción con respuestas que son "técnicamente correctas" pero no útiles para el contexto específico del usuario). El A/B testing es el método que extiende la evaluación al tráfico real de producción, con el rigor estadístico necesario para distinguir diferencias reales de ruido.

El **diseño del experimento** requiere cuatro decisiones previas al lanzamiento. Primera, la **métrica primaria**: la única métrica cuya mejora define el éxito del experimento. Debe ser una sola métrica (no "mejorar todo") para evitar ambigüedad en la interpretación de resultados y para mantener el poder estadístico del análisis. La elección depende del tipo de sistema: para un asistente de soporte técnico, la métrica primaria podría ser el porcentaje de sesiones donde el usuario no necesita reformular su pregunta (tasa de primera respuesta satisfactoria); para un sistema de QA sobre documentación, podría ser Faithfulness medida por LLM-judge sobre una muestra del tráfico. Segunda, las **métricas de guardarraíl**: métricas secundarias que no deben degradarse durante el experimento aunque no sean el objetivo principal (latencia p95, costo por query, tasa de respuestas de no-información). Tercera, el **tamaño de muestra mínimo**: calculado con un análisis de potencia estadística previo —cuántas solicitudes por variante son necesarias para detectar una diferencia de X puntos porcentuales en la métrica primaria con significancia estadística de $\alpha = 0.05$ y potencia de 0.80. Para diferencias esperadas del 5% en faithfulness (de 0.75 a 0.80), se

necesitan aproximadamente 500 solicitudes por variante. Cuarta, la **duración máxima del experimento**: aunque el tamaño de muestra no se haya alcanzado, detener el experimento después de N días para evitar el "peeking problem" (mirar los resultados antes de tiempo y declarar ganador de forma prematura).

El **mecanismo de asignación** de solicitudes o usuarios a variantes determina la comparabilidad de los grupos. La asignación aleatoria por user_id (hash del user_id módulo 100, donde 0–49 va a variante A y 50–99 va a variante B) garantiza que el mismo usuario siempre ve la misma variante durante el experimento (sticky assignment), evitando el efecto de confusión que ocurriría si el mismo usuario viera respuestas de variante A en una sesión y de variante B en otra y luego reportara feedback sobre su experiencia. La asignación se implementa con feature flags usando sistemas como LaunchDarkly, AWS AppConfig, o incluso un campo en la base de datos de usuarios si el volumen es manejable. El metadato de variante se registra junto a cada solicitud en el sistema de observabilidad (Langfuse, LangSmith) para permitir el análisis segmentado posterior.

La **evaluación automática de calidad** es el componente que convierte el A/B testing de un experimento de latencia y métricas de sistema (fáciles de medir instrumentando el código) a un experimento de calidad de las respuestas (más difícil de medir automáticamente a escala). La solución estándar es el **LLM-as-judge asíncrono**: para cada solicitud de producción (o para una muestra aleatoria del 5–10% del tráfico para reducir el costo de evaluación), ejecutar asíncronamente un evaluador LLM que puntúa Faithfulness y Answer Relevancy de la respuesta generada. Este proceso asíncrono no impacta la latencia de la respuesta al usuario porque se ejecuta en un worker separado después de que la respuesta ya fue entregada. Los scores se almacenan en la base de datos de análisis junto al identificador de variante, permitiendo calcular la distribución de scores por variante al final del experimento.

El **análisis estadístico** determina si la diferencia observada entre variantes es estadísticamente significativa. Para métricas continuas como los scores de faithfulness (valores en el rango 0.0–1.0), el test de Welch's t-test compara las medias de las dos distribuciones calculando el p-value de la diferencia. Para métricas binarias (la respuesta fue satisfactoria o no), el test de proporciones de dos muestras es el apropiado. El criterio de decisión combina significancia estadística ($p\text{-value} < 0.05$) con significancia práctica (la diferencia observada supera el umbral mínimo de efecto relevante para el sistema —por ejemplo, una diferencia de Faithfulness de 0.02 puede ser estadísticamente significativa con muestras grandes pero no tener relevancia práctica para los usuarios). Declarar ganador solo

cuando ambas condiciones se cumplen evita la trampa de optimizar diferencias estadísticamente detectables pero prácticamente irrelevantes.

Componentes del A/B testing en sistemas RAG

- **Feature flags para split de tráfico:** asignación por hash del user_id para sticky assignment; implementar con LaunchDarkly, AWS AppConfig o similar; registrar el identificador de variante (A/B) junto a cada solicitud en el sistema de observabilidad; porcentaje de tráfico configurable (50/50 para experimentos equilibrados, 90/10 para cambios arriesgados donde se quiere exposición mínima de la variante B).
- **Métricas primaria y de guardarraíl:** definir exactamente UNA métrica primaria antes del lanzamiento; guardarraíles típicos: latencia p95 < 2x variante A, costo por query < 1.2x variante A, tasa de no-respuesta < 1.5x variante A; si cualquier guardarraíl falla, pausar el experimento independientemente del estado de la métrica primaria.
- **LLM-judge asíncrono:** evaluar Faithfulness y Answer Relevancy de forma asíncrona post-respuesta; usar un LLM económico como evaluador (GPT-4o-mini, Claude 3 Haiku); muestra el 5–10% del tráfico para equilibrar costo de evaluación y representatividad; almacenar scores con variante, timestamp, user_id y query_id para análisis segmentado.
- **Cálculo de tamaño de muestra:** usar `scipy.stats.ttest_ind` con parámetro `alternative='greater'` para cálculo de potencia; minimum detectable effect definido por el equipo (5% relativo es un estándar razonable); $\alpha = 0.05$, potencia = 0.80; calcular antes del lanzamiento para saber cuándo terminar el experimento.
- **Análisis estadístico de resultados:** Welch's t-test para métricas continuas (faithfulness score), test de proporciones para métricas binarias (hit rate); calcular p-value e intervalo de confianza al 95% para la diferencia; declarar ganador cuando p-value < 0.05 Y la diferencia supera el minimum detectable effect definido previamente.
- **Rollback automático:** configurar alertas en el sistema de observabilidad que pausan automáticamente el tráfico a la variante B si la latencia p99 supera 2x variante A, si la tasa de errores aumenta más del 10%, o si cualquier guardarraíl falla; el rollback automático protege la experiencia del usuario en experimentos con configuraciones arriesgadas.

Nota del Arquitecto: El error más frecuente en A/B testing de sistemas RAG es mirar los resultados intermedios antes de alcanzar el tamaño de muestra calculado y declarar ganador de forma prematura. Con muestras pequeñas, las fluctuaciones aleatorias del score producen diferencias que parecen estadísticamente significativas pero desaparecen al alcanzar el tamaño de muestra completo. La disciplina de pre-

registrar el tamaño de muestra mínimo y la duración máxima del experimento antes del lanzamiento, y no interpretar los resultados hasta que ambas condiciones se cumplan, es lo que distingue el A/B testing riguroso del A/B testing como security theater de optimización.

El A/B testing cierra el ciclo de optimización continua del sistema RAG: diagnosticar la causa raíz de los fallos, implementar la corrección en la configuración alternativa, y validar el impacto con rigor estadístico sobre tráfico real antes de generalizar el cambio al 100% del tráfico. Este ciclo, institucionalizado como proceso del equipo, es la base del capítulo de cierre.

Módulo 6 – Capítulo 09 – Sección 06

Cierre: la optimización de RAG es un proceso continuo de medición y ajuste

Un sistema RAG lanzado a producción con métricas de calidad satisfactorias no es un sistema terminado: es un sistema que comenzó su vida de producción. El corpus evoluciona con la incorporación de nuevos documentos, actualizaciones de política y cambios de terminología en el dominio. La distribución de queries cambia a medida que los usuarios descubren nuevos casos de uso, formulan preguntas más sofisticadas o llevan el sistema a dominios para los que no fue diseñado inicialmente. Los modelos de embedding y generación se actualizan: una nueva versión de text-embedding-3 o Claude puede tener un perfil de rendimiento distinto al modelo con el que se calibró el sistema. Los SLOs de calidad se vuelven más exigentes a medida que los usuarios elevan sus expectativas. Cada uno de estos factores produce drift — una degradación silenciosa y gradual de la calidad del sistema que ningún dashboard de disponibilidad capturará porque el sistema sigue respondiendo queries, solo que cada vez con menor precisión.

Los equipos que tratan la optimización de RAG como un proyecto con un punto final —"dejamos el sistema calibrado y pasamos al siguiente"— invariablemente observan la curva que los equipos de sistemas de IA llaman "la colina y el valle": la calidad sube durante el desarrollo y el lanzamiento inicial, se estabiliza durante los primeros meses, y luego desciende silenciosamente durante el año siguiente por la acumulación de corpus drift, query distribution shift y embedding model drift no detectados. Los sistemas que mantienen su calidad en producción durante años no lo hacen gracias a técnicas más sofisticadas de recuperación, sino gracias a la institucionalización de tres procesos: un golden dataset que se actualiza trimestralmente para reflejar los nuevos patrones de uso, un pipeline de evaluación automatizado que ejecuta RAGAS y las métricas de retriever sobre ese dataset en el CI/CD de cada cambio de configuración, y dashboards de monitoring con alertas calibradas que alertan al equipo cuando Faithfulness cae por debajo de un umbral de SLO de calidad.

La institucionalización de la medición continua no es un proceso técnico, es un proceso organizacional: requiere que el equipo tenga ownership claro sobre las métricas de calidad del

sistema, con la misma visibilidad y la misma urgencia que se aplica a las métricas de disponibilidad y latencia. Un equipo que recibe una alerta a las 3am cuando el p99 de latencia supera los 5 segundos, pero no tiene ningún mecanismo de alerta cuando Faithfulness cae del 0.82 al 0.71 durante una semana porque se incorporó un nuevo lote de documentos mal procesados, tiene un sistema de monitoring incompleto. La calidad de las respuestas es el servicio que el sistema entrega a los usuarios; es igualmente merecedor de SLOs, alertas y procesos de respuesta a incidentes.

La **cadencia práctica** de optimización continua para un equipo típico podría estructurarse así: evaluación semanal automatizada del golden dataset en CI/CD con alerta si cualquier métrica cae más del 5% respecto a la baseline establecida en el lanzamiento; análisis manual mensual de 20–30 nuevos casos de error identificados en producción para actualizar la comprensión de los tipos de fallos del sistema; actualización trimestral del golden dataset incorporando nuevos tipos de queries observados en producción; A/B testing trimestral de la configuración actual vs. una configuración alternativa que explore una optimización identificada en el análisis mensual de errores; revisión semestral de los modelos de embedding y generación disponibles, evaluando si versiones más recientes producen mejoras medibles sobre el golden dataset del sistema.

Con el capítulo de optimización completado, el módulo ha cubierto el ciclo completo de un sistema RAG profesional: desde los fundamentos teóricos del Capítulo 1 hasta el mantenimiento continuo en producción de este Capítulo 9. El Capítulo 10 cierra el módulo explorando los patrones avanzados que extienden el paradigma RAG hacia sistemas de mayor autonomía y sofisticación.

"En Dios confiamos; todos los demás deben traer datos." — W. Edwards Deming, pionero del control estadístico de calidad en procesos industriales; principio igualmente aplicable a la optimización de sistemas de AI Engineering.